

推荐系统：工业架构与核心算法

RecSys-Industrial-Book

作者：刘京欣

第 2 章

推荐系统架构

2.1 混合推荐系统架构

第 1 章从整体视角介绍了推荐系统的基本概念、发展历程以及核心挑战。本章将进一步深入推荐系统的内部，从工程架构的视角，系统性地介绍工业级推荐系统的整体设计。

在大型互联网平台中，推荐系统通常需要同时支撑短视频、直播、电商、广告等多种业务。不同业务在优化目标、候选集来源以及评估指标上存在显著差异。如果为每种业务独立构建推荐系统，不仅会导致资源浪费，还难以实现跨业务的流量协调与生态平衡。因此，工业界通常采用混合推荐系统（Blended Recommendation System）架构，在统一框架下管理多业务的推荐流程。

混合推荐系统的核心思想是：在召回与排序阶段，各业务保持独立运作；而在最终分发阶段，通过混排服务将不同业务的推荐结果进行融合与流量分配。这种架构既保证了各业务推荐策略的灵活性，又实现了平台层面的全局优化。

在实际工业系统中，这种全局优化通常由混排服务承担。混排服务不仅负责融合不同业务返回的候选结果，更重要的是通过流量配额、公平竞价、多目标优化以及约束求解等机制，在平台层面完成资源分配决策，从而引导整个推荐系统向全局最优方向演化，避免陷入单一业务驱动的局部最优，最终实现用户价值、商业价值与生态价值之间的动态平衡。

2.2 级联式推荐系统

从推荐系统整体优化的视角来看，工业界的大型内容平台通常采用混合推荐系统架构，以统一管理短视频、直播、电商、广告等多个业务之间的流量分配与资源协调问题。然而，如果将视角进一步聚焦到某一个具体业务内部，例如短视频推荐、直播推荐或商品推荐，那么其核心任务则转变为：如何从海量候选内容中快速、准确地找到当前用户最感兴趣的内容。在这一场景下，工业界主流采用的是多阶段级联式推荐系统架构（Multi-stage Cascade Recommendation System Architecture）。

从第 1 章的分析可以看出，现代互联网平台通常需要同时面对亿级用户规模与十亿级内容规模所带来的双重挑战。对于一次推荐请求而言，系统既要保证推荐结果具有足够高的个性化程度，又必须在几百甚至几十毫秒内完成整个计算过程。如果直接对全量内容进行复杂模型打分，无论从计算资源还是响应延迟角度来看都是不可接受的。

 图 2.2：推荐系统级联式架构示意图（召回 10 亿 → 粗排 1 万 → 精排 500 → 重排 100 → 最终 10）

因此，工业界推荐系统逐渐演化出一种典型的“漏斗式（Funnel-based）”架构：在推荐流程的前期使用计算代价较低的方法快速缩小候选集规模，在后期逐步引入更复杂、更精确的模型进行深度评估。这种设计使得系统能够在有限的计算资源约束下，实现推荐效果与服务性能之间的最佳平衡。从本质上看，级联式推荐系

统是一种逐层筛选（Progressive Filtering）与逐层精炼（Progressive Refinement）的过程。

如图 2.2 所示，现代推荐系统通常采用四阶段级联架构，整体流程可划分为：



(1) 召回阶段：从十亿内容中找到一万个候选

召回阶段（Retrieval Stage）是整个推荐链路的入口，其主要目标是在极短时间内从海量内容库中筛选出与用户兴趣相关的候选内容。对于大型内容平台而言，候选物品规模往往达到数十亿甚至上百亿级别。工业界通常采用多路召回（Multi-channel Retrieval）策略，不同召回通路分别从不同角度刻画用户兴趣，例如：协同过滤召回、Embedding 向量召回、内容召回、热门召回、关注关系召回、序列兴趣召回等。每一路召回独立生成候选集合，最终汇总形成约 1 万至数万规模的候选池。在召回之后，系统通常还会执行一个过滤（Filtering）模块，用于完成内容去重、已曝光内容过滤、风险内容过滤、用户黑名单过滤、合规与安全策略过滤等。

(2) 粗排阶段：用低成本模型快速筛选

经过召回过滤阶段之后，候选集合规模虽然已经大幅下降，但通常仍然包含数千到数万个候选内容。此时如果直接使用复杂深度模型进行打分，依然会产生巨大的计算开销。因此系统会引入粗排阶段（Pre-Ranking Stage），利用计算代价较低的模型进行快速筛选。粗排模型通常具有模型结构简单、特征数量较少、推理速度快等特点。工业界常见的粗排模型包括：双塔模型（Two-Tower Model）、浅层 DNN、多塔模型等。粗排阶段还经常承担业务流量调控职责，例如长尾内容扶持、新内容冷启动、内容垂类配额控制、创作者流量保护等。

(3) 精排阶段：决定推荐效果的核心环节

如果说召回决定了系统是否能够"找到正确的内容"，那么精排阶段（Ranking Stage）则决定了系统能否"把最合适的内容排在最前面"。精排是整个推荐系统中最核心、最重要的模块，也是算法创新最活跃的部分。常见精排模型包括：Wide & Deep、DeepFM、xDeepFM、DIN、DIEN、DCN、Transformer-based Ranking Model、多任务学习模型（MMOE、PLE）等。现代推荐系统往往同时优化 CTR、CVR、观看时长、互动率、留存率等多个目标。经过精排后，系统通常会保留约 100 个左右的候选内容进入重排阶段。

(4) 重排阶段：从单点最优到列表最优

前面的召回、粗排和精排，本质上都在解决一个问题："这个内容对用户有多大吸引力？"然而，当多个内容共同出现在推荐列表中时，仅仅优化单个内容的得分并不足够。例如：连续出现 10 条篮球视频、全部来自同一个创作者等。因此，重排阶段（Re-Ranking Stage）关注的不再是单个物品，而是整个推荐列表的质量，包括多样性、新颖性、公平性、品类均衡、创作者生态治理等。重排实际上是在求解一个序列级优化问题（List-wise Optimization Problem）。

综上所述，多阶段级联推荐架构本质上是一种"先召回、再筛选、后优化"的分层决策体系。通过这种由粗到精、层层过滤的设计思想，工业界成功解决了海量内容场景下推荐效果与系统性能之间的矛盾。

2.3 推荐系统的离线架构与在线架构

对于很多初学者而言，推荐系统往往等同于 CTR 预估模型、召回模型或者排序模型。然而在工业界，一个推荐模型只是推荐系统中的一个组成部分。一个真正可运行的推荐系统，还需要解决数据采集、样本构建、特征存储、模型训练、模型部署、在线推理以及服务稳定性保障等一系列工程问题。

笔记

为什么推荐系统叫做"推荐系统"，而不仅仅叫做"推荐模型"？

答案在于：推荐模型只是系统中的一个组件，而推荐系统本质上是一套围绕用户行为数据不断运转、自我迭代的数据飞轮（Data Flywheel）。

2.3.1 离线架构

推荐系统的离线部分主要负责数据生产、样本构建、特征计算、索引维护以及模型训练等工作。从数据流的角度来看，离线架构实际上承担着"学习用户兴趣"的职责。

推荐系统的离线架构主要包括推荐数据的获取与发送、用户反馈上传、推荐样本拼接、特征生成以及模型离线训练等环节。整个流程包含两个关键分支：

分支一（标签流）：当用户与推荐物品发生交互行为（如点击、点赞、分享等）后，客户端会立即将行为日志上报至埋点网关。埋点网关对数据进行鉴权、校验、限流及格式标准化处理后，将其以 Kafka 消息的形式发布。下游的"反馈标签消费者"会实时消费这些消息，并将用户反馈以 Key-Value 的形式写入 Redis 缓存。其中，Key 通常由请求 ID + 物品 ID 构成。

分支二（特征流）：在用户每次发起推荐请求时，服务端通过推荐 Router 调用下游推荐系统。在召回与排序过程中，系统会从索引服务中获取物品的完整特征，并将本次请求的用户特征、物品特征及上下文信息打包为 Kafka 消息发送出去。下游的"样本拼接消费者"会消费该消息，并在一个固定（例如 20 分钟）或动态设定的拼接窗口内，尝试从 Redis 中查询对应的用户反馈。若在窗口期内成功匹配到反馈标签，则将特征与标签拼接为完整的训练样本（`user`，`item`，`context`，`label`），并写入新的 Kafka Topic，供离线模型训练任务消费。

 图 2.3：推荐系统离线架构（推荐 Router → 索引服务 → Redis → 样本拼接消费者 → 训练服务）

在图 2.3 所示的推荐系统离线架构中，涉及若干关键技术组件：

- **Kafka**：高吞吐、分布式的流式消息队列系统。在推荐系统中，Kafka 主要负责承载用户行为日志（如曝光、点击、点赞、停留时长等）的实时传输。
- **索引服务**：特指物品侧（Item-side）的特征索引服务，即根据 Item ID 快速获取对应物品的完整特征信息。该服务也被称为正排索引（Forward Index），与召回阶段使用的倒排索引或向量索引不同。
- **RPC 调用**：远程过程调用（Remote Procedure Call），允许调用方以本地函数调用的方式访问远程服务。推荐系统中各服务之间通常通过 RPC 进行通信。
- **Redis 集群**：基于内存的高性能键值存储系统，常用于缓存用户特征、物品特征、实时统计指标以及样本拼接过程中的中间结果。
- **样本拼接服务**：负责将用户行为日志、用户特征、物品特征以及上下文信息等来自不同数据源的异构数据进行关联与融合，最终生成标准化训练样本（`user, item, context, label`）。

读者可能会注意到，图 2.3 中离线训练所依赖的数据源是实时 Kafka 消息流，而非传统推荐学习中常见的静态用户—物品交互矩阵。这一设计源于工业级推荐系统对于数据规模与实时性的双重要求。越来越多的工业级推荐系统开始采用基于实时数据流的训练范式。当然，并非所有业务场景都必须采用流式训练，例如广告推荐场景中仍然常采用基于 HDFS 等离线存储的批处理方式。

用户侧特征存储

推荐系统还需要长期维护用户画像（User Profile）以及用户行为序列。用户行为日志通常会存储于 HDFS、Hive、ClickHouse 等分布式存储系统中。由于原始行为序列长度往往非常长，工业界通常会离线构建不同粒度的用户特征，例如：最近 50 条行为序列、最近 7 天行为序列、最近 30 天兴趣统计、长期兴趣画像等。这些特征最终会被写入用户特征中心（User Feature Store），供在线推荐服务实时读取。从某种意义上来说，用户特征中心承担着推荐系统“长期记忆”的角色。

物品侧特征存储与索引更新

与用户侧特征类似，推荐系统同样需要维护海量物品的特征信息。这些信息通常以（`item_id, feature`）的形式存储于分布式索引系统中。由于物品状态会持

续发生变化，工业界通常采用 Batch 全量更新和 Streaming 实时更新相结合的方式维护索引。全量更新通常由离线任务轮询周期性执行，实时更新则由 Kafka 消息流进行触发。这种"Batch + Streaming"的双重更新模式既保证了索引的一致性，又保证了内容的新鲜度（Freshness）。

2.3.2 在线架构

在线推荐系统的核心任务是在极短时间内完成一次推荐决策。它直接面向用户请求运行，主要负责从海量候选物品中快速筛选出用户当前最可能感兴趣的 Top-K 结果。与离线架构相比，在线推荐系统更加关注三个核心问题：**低延迟、高可用、高吞吐**。

对于大型互联网平台而言，每一次用户下滑页面、刷新信息流或进入推荐页面，都会触发一次新的推荐请求。推荐系统通常存在严格的耗时约束，例如整体响应时间需要控制在数百毫秒以内。除了低延迟之外，高可用性同样重要。工业界通常会在推荐系统中设计一系列兜底机制（Backup Mechanism），例如当召回服务异常、排序服务超时或者模型推理服务不可用时，系统可以自动切换至预先准备好的高热内容池、运营内容池或者缓存推荐结果。

推荐系统还需要面对巨大的算力成本压力。由于用户访问行为具有明显的潮汐特征，推荐系统的请求量通常会呈现显著的波峰与波谷现象。工业界通常会根据实时流量情况进行动态算力分配（Dynamic Resource Allocation）。当系统处于低负载状态时，可以启用更复杂的召回策略、更大的候选集合以及更精细的排序模型；而在流量高峰阶段，则适当降低部分非核心计算开销。

2.4 降级推荐系统

当系统处于流量高峰期、计算资源紧张或部分核心服务发生异常时，如果仍然对所有请求执行完整的召回、粗排、精排与重排流程，往往会导致系统响应时间急剧上升，甚至出现服务不可用的情况。因此，工业界通常会设计专门的降级推荐系统（Degraded Recommendation System）。

定义 2.2

在实时推荐系统正常运行时，将当前用户请求中排序结果中的 Top 10 至 Top 30 物品缓存至 Redis 中。当系统触发降级时，直接从 Redis 中读取这些预先计算好的推荐结果，并选取其中的前 10 个物品返回给用户，从而避免执行完整的召回与排序流程，大幅降低系统的计算开销和响应延迟。如果 Redis 缓存中还有没用完的缓存物品，下一次这个用户触发降级请求的时候还可以继续使用。

降级推荐系统与实时推荐系统的协同决策流程如下：

1. 用户请求到达推荐 Router 后，Router 首先通过 RPC 调用降级决策服务（Degradation Decision Service）。
2. 降级决策服务根据当前系统状态返回是否触发降级的决策标识（Flag）。Flag=0 表示继续使用实时推荐系统；Flag=1 表示进入降级模式。
3. 推荐 Router 根据返回结果选择后续处理链路：当 Flag=0 时，请求实时推荐系统；当 Flag=1 时，请求降级推荐系统。

降级决策服务可以基于规则控制（如 CPU 利用率、QPS、平均响应时间等指标设置固定阈值），也可以采用强化学习等智能决策方法。例如，推荐 Router 及下游服务在 CPU 利用率低于 75% 的安全线之内，能提前估计出整个推荐链路能够承载的平均 QPS 是 2 万。在高峰期时 QPS 往往会高于 2 万，推荐 Router 就可以随机选取 2 万之外的用户请求，让其路由到降级推荐系统。

DCAF：动态算力分配框架

本节介绍一种经典的动态算力分配框架——DCAF（Dynamic Computation Allocation Framework）[2]。其核心思想是：

定义 2.3

打破传统推荐系统对所有请求"一视同仁"的算力分配方式，而是根据每个请求的潜在价值动态分配计算资源。

DCAF 将资源分配问题建模为一个带约束的背包问题（Knapsack Problem）：

$$\begin{aligned} \max_{x_{ij}} \quad & \sum_{i=1}^N \sum_{j=1}^M x_{ij} Q_{ij} \\ \text{s.t.} \quad & \sum_{i=1}^N \sum_{j=1}^M x_{ij} q_j \leq C, \quad \sum_{j=1}^M x_{ij} \leq 1, \quad x_{ij} \in \{0, 1\} \end{aligned} \quad (2.1)$$

其中： $x_{ij} = 1$ 表示请求 i 选择动作 j ； Q_{ij} 表示请求 i 在动作 j 下能够获得的预期收益； q_j 表示动作 j 所对应的计算资源消耗。

通过拉格朗日对偶分析，DCAF 证明：在收益随计算资源增加而递增、但边际收益递减的条件下，最优策略可简化为：

$$\arg \max_j (Q_{ij} - \lambda q_j) \quad (2.2)$$

即对于每个请求，只需选择使 $Q_{ij} - \lambda q_j$ 最大的动作即可。其中 λ 为与全局资源预算相关的拉格朗日乘子，可通过二分搜索等方法高效求解。

DCAF 在线服务系统主要由两个模块组成：

- **请求价值估计器（Offline Estimator）**：利用历史日志训练轻量级价值评估模型，实时预测请求在不同动作下的预期收益 Q_{ij} 。在广告场景中通常采用 eCPM 衡量。

- **策略执行器 (Online Decision Maker)**：根据价值估计结果以及当前最优 λ 参数，为每个请求动态选择对应动作。

此外，DCAF 还引入了 MaxPower 机制。当系统遭遇流量突发时，通过 PID 控制器实时监控 GPU 利用率、响应时间以及服务失败率等指标，并动态调整单请求允许使用的最大算力。论文实验表明，DCAF 已成功应用于淘宝展示广告系统，在广告收入 (RPM) 和点击率 (CTR) 基本保持不变的前提下，系统能够降低约 20% 的 GPU 资源消耗。

降级推荐系统内部通常包含三个步骤：**降级 Redis 缓存召回**（直接从缓存中读取历史推荐结果）、**过滤规则**（去重、风控、内容安全）、**结果截断**（按 Top-K 截取）。

2.5 作者侧推荐系统

在绝大多数工业级推荐系统中，推荐服务通常围绕用户请求展开。这类以用户需求为中心构建的推荐架构，通常被称为用户侧推荐系统 (User-side Recommendation System)。然而，用户侧推荐系统在长期运行过程中往往会面临流量向头部内容过度集中问题，形成典型的“马太效应 (Matthew Effect)”。

如果从平台生态建设的视角来看，仅依赖用户侧推荐系统则可能带来新的问题。推荐系统本质上连接的是内容消费者与内容生产者。如果平台中的绝大部分流量持续流向头部创作者，而中腰部及新创作者长期无法获得成长机会，那么内容供给将逐渐趋于单一化，创作者活跃度也可能持续下降。

因此，对于内容平台而言，推荐系统实际上需要同时优化两个目标：

- **用户价值 (Consumer Value)**：提升用户满意度、观看时长、留存率等核心消费指标；
- **生产者价值 (Producer Value)**：促进创作者成长，提高内容供给质量与生态繁荣度。

近年来工业界逐渐提出并探索一种与用户侧推荐系统相对应的推荐范式，即作者侧推荐系统（Creator-side Recommendation System），也被称为对偶推荐系统（Dual Recommendation System, DualRec）[1]。

定义 2.4

首先从内容和创作者的视角出发，通过作者推荐系统找到当前在线同时对该内容感兴趣的用户。进而通过用户侧推荐系统与作者侧推荐系统相互结合与补充，将其纳入到一个完整的推荐框架之中，在提升用户体验的同时，也能促进内容生态的长期繁荣与可持续发展。

在作者侧推荐系统中，推荐过程不再以用户作为起点，而是以内容或创作者作为起点展开。对于一个待分发的物品（Item），系统首先从在线活跃用户集合中召回可能对该内容感兴趣的用户，并进一步通过排序模型评估用户与物品之间的匹配程度，最终得到该物品对应的一组目标用户集合。

笔记

传统推荐系统解决的是"给用户推荐什么内容"

作者侧推荐系统解决的是"给内容寻找哪些潜在用户"

完成用户召回与排序后，对于每个用户 u ，将所有通过作者侧推荐系统匹配到该用户的物品聚合形成候选集合：

$$D_u = \{item_1, item_2, \dots, item_n\}$$

(2.4)

随后，系统以用户 ID 作为 Key，将对应的候选集合 D_u 以 Key-Value 的形式写入 Redis 缓存。对于作者侧推荐结果数量仍未达到用户日消费上限 Q 的用户，即

满足 $|D_u| < Q$ ，系统会将其加入在线用户池中。DualRec 还会引入用户活跃度预测服务，对用户未来一段时间内是否仍将保持在线状态进行预测。

作者侧推荐结果最终会与传统用户侧推荐系统进行融合，共同参与最终推荐列表的生成。系统通过保量（Quota Protection）、Boost 加权、混排融合（Mix Ranking）等策略，将作者侧推荐结果与用户侧推荐结果进行联合排序，最终生成推荐给用户的物品列表。前者回答的是"用户想看什么"，后者回答的是"哪些内容值得被更多用户看到"。

2.6 系统日志与数据

推荐系统中的日志与数据是提升推荐模型效果的核心基础。从某种意义上说，推荐系统的竞争，本质上也是数据质量与数据利用能力的竞争。尽管我们习惯将 CTR 预估模型、排序模型乃至大模型推荐框架统称为"AI 模型"，但归根结底，它们本质上仍然是高度依赖数据驱动的数据统计学习系统（Data-Driven Statistical Learning System）。

与早期基于用户-物品评分矩阵（如 MovieLens 数据集）的推荐方法不同，现代工业级推荐系统所面对的数据环境更加复杂。一方面，可利用的数据维度极其丰富，包括用户画像、物品特征、上下文信息、行为反馈信息等。另一方面，用户兴趣本身也具有明显的动态演化特征。同时，工业场景中的数据规模通常达到 TB 级甚至 PB 级，海量数据带来了数据质量、反馈稀疏、反馈延迟、分布偏移等挑战。

2.6.1 流式数据与离线数据

从数据生产方式来看，工业界推荐系统中的数据通常以两种形式存在：实时数据流（Streaming Data）与离线批量数据（Batch Data）。

近线更新（Nearline Update）：通常指基于实时行为流持续更新用户特征或物品特征，例如最近点击序列、短期兴趣 Embedding、实时热度统计等，而模型参数本身并不发生变化。

在线更新 (Online Learning)：则更进一步，模型参数会随着新反馈到来而实时学习与更新。例如在线 Logistic Regression、FTRL 等在线学习框架。

在实际工业系统中，近线特征更新远比在线参数更新更为常见。原因在于在线学习虽然能够获得更快的反馈响应速度，但同时也带来了模型稳定性、参数漂移以及线上风险控制等问题。

2.6.2 Hive 与推荐系统数据仓库

在工业实践中，上述离线数据通常以 Hive 表的形式组织存储。Apache Hive 是构建在 Hadoop 生态之上的数据仓库系统，其核心作用是为海量数据提供类似 SQL 的查询能力。借助 Hive SQL (HQL)，开发者无需编写复杂的 MapReduce 程序，便能够直接对存储于 HDFS 中的 PB 级数据进行查询与分析。对于推荐算法工程师而言，Hive SQL 几乎是一项必备技能。

2.6.3 推荐系统中的核心日志

以短视频推荐场景为例，推荐系统中的日志通常围绕三个核心对象展开：用户 (User)、物品 (Item)、用户与物品的交互 (Interaction)。因此，工业界推荐系统最常见的数据表通常可以归纳为三类：

1. 用户侧基础信息日志 (User Profile Log)：通常记录用户的静态画像与设备属性，通常每日快照一次。字段包括：user_id、device_id、age、gender、interest_tags、os、app_version、is_new_user 等。

表 2.1: 用户侧基础信息日志示例

user_id	device_id	age	gender	interest_tags	os	app_version	i
10086	d7a3b2c1	28	female	# 美食,# 旅行	Android 12	v3.4.1	
20240	e9f8d7c6	19	male	# 游戏,# 科技	iOS 16	v3.4.0	
30571	a1b2c3d4	35	male			v3.4.2	

user_id	device_id	age	gender	interest_tags	os	app_version	i
				# 财经,# 新闻,# 汽车	Android 13		
40892	f5e4d3c2	24	female	# 美妆,# 穿搭,# 健身	iOS 17	v3.4.1	
51203	9z8y7x6w	42	female	# 教育,# 亲子,# 家居	Android 11	v3.3.9	
62314	m2n3o4p5	17	male	# 动漫,# 电竞,# 短视频	iOS 15	v3.4.0	

2. 物品侧基础信息日志（Item Metadata Log）：描述物品内容及其元数据，由内容理解系统或创作者后台生成。

表 2.2: 短视频基础信息日志示例

video_id	author_id	author_device_id	duration_sec	tags	like_cnt	comm
v12345	70349	z5f8c5k0	45	# 萌宠,# 搞笑	12500	8
v67890	81426	q2w9e7r4	28	# 旅行,# 海岛	3200	1
v24680	93571	m3n8b2v6	62	# 知识,# 科普	8900	1

video_id	author_id	author_device_id	duration_sec	tags	like_cnt	comm
v13579	64802	t7y4u9i1	18	# 美 妆,# 教 程	21000	2
v97531	55128	j6h3g9f2	55	# 剧 情,# 情 感	45600	3
v86420	47293	x8c4v6b1	12	# 音 乐,# 卡 点	6700	3

3. 用户—物品交互日志（Interaction Log）：记录用户与视频在不同时间粒度下的行为，是推荐模型训练的核心信号来源。

表 2.3：用户-短视频交互日志示例（请求粒度）

req_id	user_id	device_id	video_id	author_id	author_device_id	watch
r98765	10086	d7a3b2c1	v12345	70349	z5f8c5k0	42
r87654	20240	e9f8d7c6	v67890	81426	q2w9e7r4	28
r76543	30571	a1b2c3d4	v24680	93571	m3n8b2v6	2
r65432	40892	f5e4d3c2	v13579	64802	t7y4u9i1	18
r54321	51203	9z8y7x6w	v97531	55128	j6h3g9f2	55
r43210	62314	m2n3o4p5	v86420	47293	x8c4v6b1	5

2.6.4 大模型驱动的内容理解数据

仅依赖传统元数据仍然存在明显局限。例如：视频时长、视频标签、作者信息、点赞数这些字段虽然能够描述内容的一部分属性，但往往难以刻画内容真正的语义信息。随着大语言模型（LLM）与多模态大模型（VLM）的快速发展，推荐系统开始逐渐从"行为驱动推荐"向"行为 + 语义双驱动推荐"演进。

在短视频场景中，LLM 可以分析 ASR 文本、总结评论区观点、自动生成主题标签，VLM 可以理解视觉内容、生成统一的多模态 Embedding。工业界通常会构建一张由大模型增强的内容元数据表：

表 2.4：LLM/VLM 增强的短视频元数据表示例

video_id	asr_summary	topic	category	comment_summary	visual_emb
v12345	一只橘猫被逗猫棒逗得疯狂扑跳，全程搞笑不断。	萌宠互动	宠物	"猫咪反应太可爱了！笑到肚子疼"	[0.82, -0.31, 0.45, ...]
v67890	探访马尔代夫私人岛屿，展示日落海滩与水上别墅。	海岛旅行	旅行	"想去这个地方！摄影太美了"	[-0.12, 0.67, -0.23, ...]
v24680	用生活实例解释牛顿三大定律，通俗易懂。	物理科普	知识	"讲得通俗易懂，期待下一期"	[0.55, 0.28, 0.91, ...]
v13579	演示新手如何画出自然日常眼妆，步骤清晰。	美妆教程	美妆	"步骤清晰，已收藏反复看"	[0.33, -0.44, 0.76, ...]
v97531		亲情	剧情		

video_id	asr_summary	topic	category	comment_summary	visual_emb
	讲述单亲父亲独自抚养女儿的温情故事。			"看哭了，真实得让人心疼"	[-0.61, 0.19, -0.52, ...]
v86420	卡点舞蹈配合热门BGM，节奏感极强。	舞蹈卡点	音乐	"节奏感绝了！BGM求链接"	[0.70, 0.88, -0.15, ...]

通过将此类由 LLM 与 VLM 生成的语义增强数据与用户行为日志深度融合，推荐系统的能力边界得以显著拓展：不仅能够基于历史交互"知道用户喜欢什么"，更能借助深层次的内容理解"洞察内容为何被喜欢"。这种从行为信号驱动向语义理解驱动的范式跃迁，使得推荐系统在捕捉用户兴趣、识别内容价值以及建模用户-内容匹配关系等方面更加精准、鲁棒且具备可解释性。

综上所述，日志与数据构成了推荐系统的基础设施层。从用户行为采集、离线数据仓库建设，到训练样本构建，再到大模型驱动的内容理解，整个数据体系持续为推荐模型提供学习所需的"燃料"。随着 LLM 与 VLM 技术的发展，推荐系统正在逐渐突破传统行为建模的边界，从"根据用户过去点击过什么进行推荐"，演进到"理解用户为什么喜欢某类内容以及内容本身表达了什么"。这种从行为信号驱动（Behavior-driven）向语义理解驱动（Semantic-driven）的演进，正在成为下一代智能推荐系统的重要发展方向。

2.7 本章小结

本章在上一章推荐系统概述的基础上，进一步从系统工程的视角介绍了工业级推荐系统的整体架构。首先，从多业务混合推荐系统架构出发，介绍了推荐系统在实际业务中的复杂组织形式，并分析了混排模块在多业务流量分配与结果融合中的作用机制。随后，聚焦于单一业务场景下的经典级联式推荐架构，系统梳理了召回、粗排、精排以及重排等核心模块之间的协同关系。

在此基础上，本章进一步介绍了推荐系统的离线架构与在线架构。离线架构主要关注用户行为数据的采集、样本构建、特征生成、索引维护以及模型训练等流程；在线架构则更加关注推荐服务的低延迟、高并发、高可用等工程问题。随后介绍了工业界常见的降级推荐系统与作者侧推荐系统（DualRec）。降级推荐系统通过动态算力分配与缓存机制保障推荐服务在高并发场景下的稳定运行；作者侧推荐系统则从内容生产与创作者生态的角度出发，利用推荐系统的对偶性实现用户价值与内容生态价值的协同优化。

最后，本章介绍了推荐系统中的日志与数据体系，包括用户侧基础信息日志、物品侧基础信息日志、用户—物品交互日志以及由大语言模型和多模态大模型生成的内容理解数据等。通过本章的学习，读者不仅能够理解推荐系统各个核心模块的功能与职责，还能够从整体视角认识工业级推荐系统的数据流、服务流以及模型流是如何协同运作的。

2.8 参考文献

[1] Xiaoshuang Chen et al. "Creator-Side Recommender System: Challenges, Designs, and Applications". In: Companion Proceedings of the ACM on Web Conference 2025. 2025, pp. 162–170.

[2] Biye Jiang et al. "DCAF: A Dynamic Computation Allocation Framework for Online Serving System". In: arXiv preprint arXiv:2006.09684 (2020).

✱ ✱ ✱

[下一章：第3章 推荐系统评价体系 →](#)